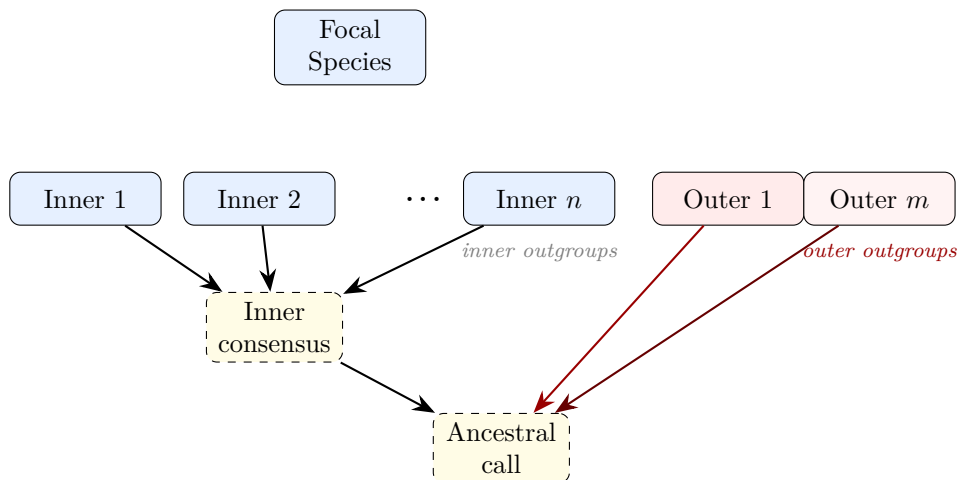


ancify

Ancestral Allele Polarization Pipeline for Any Species

A species-agnostic, config-driven Python package
for inferring ancestral alleles using outgroup alignments



Contents

1	Introduction	3
1.1	Why Polarize Variants?	3
1.2	The Outgroup Approach	3
1.3	What This Package Does	3
2	Biological Background	3
2.1	Inner vs. Outer Outgroups	3
2.2	Net Alignments and the AXT Format	4
3	Installation	4
3.1	From Source	4
3.2	With Evaluation Dependencies	4
3.3	Core Dependencies	5
4	Package Architecture	5
4.1	Module Layout	5
4.2	Three-Phase Pipeline	5
5	Configuration	6
5.1	Configuration Reference	6
5.2	Key Configuration Fields	7
5.3	Pattern Placeholders	7
6	Phase 1: Coordinate Projection	7
6.1	Purpose	7
6.2	Algorithm (<code>ancify.project</code>)	7
6.3	Output	8
7	Phase 2: Ancestral State Inference	8
7.1	The Generalized Algorithm	8
7.2	Core Function	9
7.3	The <code>majority_vote()</code> Function	10
7.4	Output Encoding	10
8	Phase 3: Evaluation	10
8.1	Available Metrics	10
8.2	Output	11
9	Command-Line Interface	11
9.1	Available Commands	11
9.2	Global Options	11
10	Worked Example: Human (hg38) BCGM	12
10.1	Phylogenetic Context	12
10.2	Configuration	12
10.3	Input Data	13
10.4	Running the Pipeline	13
10.5	Evaluation Results	13
11	Adapting to Other Species	14
11.1	What You Need	14
11.2	Choosing Inner vs. Outer Outgroups	14

11.3 Example: Mouse	14
11.4 Example: <i>Drosophila melanogaster</i>	14
11.5 Creating a Chromosome-Lengths File	15
12 Using the Output Downstream	15
12.1 Looking Up an Ancestral Allele	15
12.2 Polarizing a VCF File	15
12.3 Filtering by Confidence	16
12.4 Programmatic API	16
13 Theoretical Considerations	17
13.1 Misinference from Incomplete Lineage Sorting	17
13.2 Back Mutations and Convergence	17
13.3 Alignment Quality	17
13.4 The <code>min_inner_freq</code> / <code>min_outer_freq</code> Parameters	17
14 Troubleshooting	17
15 Complete Module Reference	18
A Quick Reference Card	19
B Comparison: General Package vs. Original Scripts	19

1 Introduction

1.1 Why Polarize Variants?

In population genetics, most analyses of the **site frequency spectrum** (SFS) require knowing whether an allele is *ancestral* (inherited from the common ancestor) or *derived* (arose by mutation). Without polarization, the SFS is *folded*—one cannot distinguish a rare derived allele from a rare ancestral allele—which limits the power of tests for selection, demographic inference, and mutation-rate estimation.

Polarization is required for:

- **Unfolded SFS:** used by *∂a∂i*, *moments*, *mushi*, and similar demographic inference tools.
- **Tests of selection:** McDonald–Kreitman, Fay & Wu’s H , D_n/D_s , etc.
- **Ancestral recombination graphs:** correct orientation of mutations on genealogies.
- **Mutation spectra:** directional mutation-rate estimation requires knowing which allele is ancestral.

1.2 The Outgroup Approach

The standard approach compares the focal species to one or more **outgroup** species. If an outgroup carries allele X and the focal species is polymorphic for X and Y , parsimony suggests X is ancestral. Using *multiple* outgroups reduces misinference from back-mutations, convergent evolution, and incomplete lineage sorting (ILS).

1.3 What This Package Does

The *ancify* package generalises this logic into a species-agnostic, config-driven pipeline:

1. **Phase 1 — Projection:** converts pairwise net-AXT alignments from any number of outgroup species into FASTA files in focal-species coordinates.
2. **Phase 2 — Ancestral calling:** infers the ancestral allele at every position using a two-tier inner/outer outgroup voting scheme with case-encoded confidence.
3. **Phase 3 — Evaluation** (optional): compares the calls against a reference ancestral sequence and/or VCF variant data.

The pipeline works for **any focal species** for which pairwise net-AXT alignments to outgroup species are available (typically from the UCSC Genome Browser).

2 Biological Background

2.1 Inner vs. Outer Outgroups

The pipeline partitions outgroup species into two tiers:

Inner outgroup One or more *closely related* species. A majority vote among them produces the **inner consensus**. Because these species diverged recently from the focal species, most of their genome is identical to the true ancestral state.

Outer outgroup One or more *distantly related* species. They serve as an independent check: if the inner consensus agrees with the outer outgroup, the call is made with high confidence.

Example: Human Polarization

For *Homo sapiens* (hg38):

- Inner: bonobo (~6 Mya), chimpanzee (~6 Mya), gorilla (~9 Mya)
- Outer: rhesus macaque (~25 Mya)

If all three great apes agree on allele **A** and macaque also carries **A**, the ancestral call is **A** with high confidence.

Example: Drosophila Polarization

For *D. melanogaster* (dm6):

- Inner: *D. simulans*, *D. sechellia*
- Outer: *D. yakuba*

Same algorithm, different species tree.

2.2 Net Alignments and the AXT Format

The pipeline uses **net alignments** from the UCSC Genome Browser—one-to-one best-chain pairwise alignments stored in **AXT format**. Each alignment block consists of four lines:

1. Header: index, target chromosome, start, end, query chromosome, start, end, strand, score.
2. Target (focal) aligned sequence (with gap characters -).
3. Query (outgroup) aligned sequence.
4. Blank separator.

These files can be downloaded from UCSC for hundreds of species pairs.

3 Installation

3.1 From Source

```
1 cd ancify/  
2 pip install .
```

This installs the **ancify** command-line tool and the **ancify** Python package.

3.2 With Evaluation Dependencies

The evaluation phase (Phase 3) requires **scikit-allel** and **matplotlib**:

```
1 pip install '.[evaluate]'
```

3.3 Core Dependencies

Table 1: Package dependencies.

Package	Purpose	Required?
pyyaml	Configuration file parsing	Yes
numpy	Array operations in evaluation	Yes
scikit-allel	VCF reading (Phase 3)	Optional
matplotlib	Plotting (Phase 3)	Optional

4 Package Architecture

4.1 Module Layout

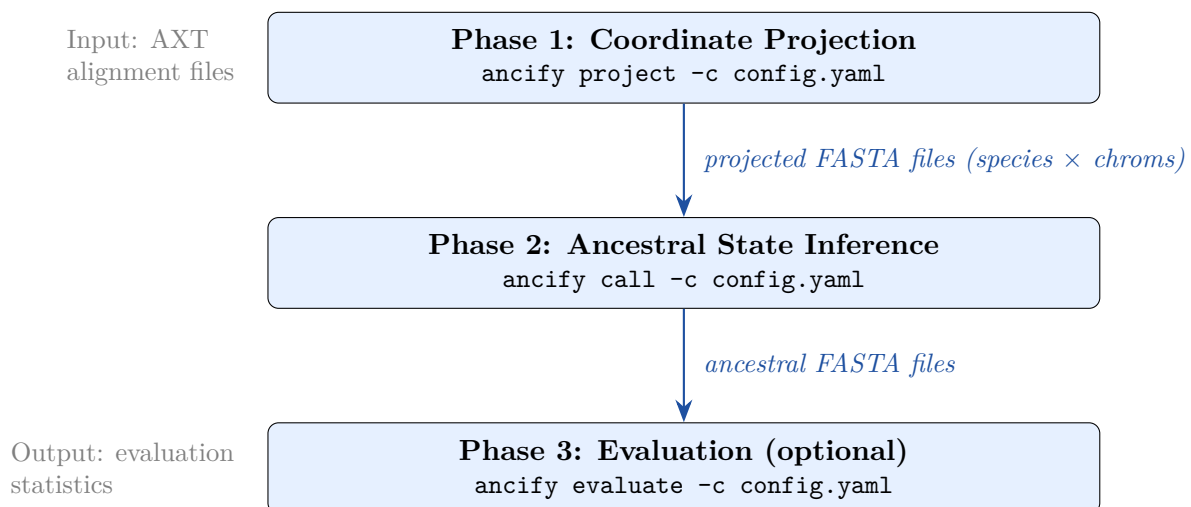
```

1 ancify/                                # project root
2 |-- pyproject.toml                     # build metadata & dependencies
3 |-- example_configs/
4 |   |-- hg38_bcgm.yaml                 # human worked example
5 |   |-- mouse_example.yaml             # mouse hypothetical example
6 |   |-- drosophila_example.yaml
7 |-- ancify/                            # importable Python package
8 |   |-- __init__.py
9 |   |-- __main__.py                   # python -m ancify
10 |   |-- cli.py                        # command-line interface
11 |   |-- config.py                     # YAML config loading & validation
12 |   |-- utils.py                      # FASTA I/O, majority_vote, helpers
13 |   |-- project.py                    # Phase 1: coordinate projection
14 |   |-- ancestral.py                  # Phase 2: ancestral state inference
15 |   |-- evaluate.py                   # Phase 3: evaluation & QC

```

Listing 1: Package file structure.

4.2 Three-Phase Pipeline

Figure 1: Three-phase pipeline, each invocable independently or together via `ancify run`.

5 Configuration

All pipeline behaviour is controlled by a single **YAML configuration file**. Generate a template with:

```
1 ancify init -o config.yaml
```

5.1 Configuration Reference

```
1 # Informal label for the focal species.
2 focal_species: human
3
4 # Tab-separated file: chrom_name <TAB> length [<TAB> ...]
5 chromosome_lengths: chromoLens.txt
6
7 # Optional: restrict to specific chromosomes.
8 # If omitted, all entries in the lengths file are used.
9 chromosomes:
10   - chr1
11   - chr2
12   - chrX
13
14 outgroups:
15   # Inner: closely related species (majority vote).
16   inner:
17     - name: bonobo
18       alignment: hg38.panPan3.net.axt.gz
19     - name: chimp
20       alignment: hg38.panTro6.net.axt.gz
21     - name: gorilla
22       alignment: hg38.gorGor6.net.axt.gz
23   # Outer: distantly related species (independent check).
24   outer:
25     - name: macaque
26       alignment: hg38.rheMac10.net.axt.gz
27
28 # Intermediate files go here (projected/ subdirectory).
29 work_dir: .
30
31 # Final ancestral FASTA files go here.
32 output_dir: ./ancestral_calls
33
34 # Minimum allele count for the majority-vote consensus.
35 min_inner_freq: 1
36 min_outer_freq: 1
37
38 # Parallel workers (one chromosome per worker).
39 num_cpus: 24
40
41 # Optional evaluation block.
42 evaluation:
43   reference_dir: ./reference_ancestral/
44   # {chrom} = full name, {chrom_id} = without "chr" prefix
45   reference_pattern: "ancestor_{chrom_id}.fa"
46   vcf_dir: ./vcf/
47   vcf_pattern: "variants.{chrom}.vcf.gz"
```

Listing 2: Annotated configuration file.

5.2 Key Configuration Fields

Table 2: Configuration fields and their meaning.

Field	Required	Description
<code>focal_species</code>	Yes	Label for the focal species (cosmetic).
<code>chromosome_lengths</code>	Yes	Path to tab-separated lengths file.
<code>chromosomes</code>	No	Subset of chromosomes to process; defaults to all.
<code>outgroups.inner</code>	Yes	List of inner outgroup species (name + alignment path).
<code>outgroups.outer</code>	Yes	List of outer outgroup species (name + alignment path).
<code>work_dir</code>	No	Working directory for projected sequences (default: <code>.</code>).
<code>output_dir</code>	No	Output directory for ancestral FASTAs (default: <code>./ancestral_calls</code>).
<code>min_inner_freq</code>	No	Minimum count for inner majority vote (default: 1).
<code>min_outer_freq</code>	No	Minimum count for outer majority vote (default: 1).
<code>num_cpus</code>	No	Parallel workers (default: 4).
<code>evaluation</code>	No	Optional block for Phase 3 settings.

5.3 Pattern Placeholders

Evaluation filename patterns support two placeholders:

- `{chrom}` — the full chromosome name as it appears in the config (e.g. `chr1`, `2L`).
- `{chrom_id}` — the name with any leading `chr` prefix stripped (e.g. `1`, `X`).

6 Phase 1: Coordinate Projection

6.1 Purpose

Transforms each outgroup’s pairwise alignment (AXT format) into a FASTA file where every position corresponds to a position in the focal-species reference. Unaligned positions are filled with N.

6.2 Algorithm (*ancify.project*)

1. Load focal-species chromosome lengths from the config.
2. For each outgroup $\times$ chromosome combination:
 - (a) Allocate a byte array of length L (chromosome length), pre-filled with N.
 - (b) Stream through the AXT file. For each alignment block matching the target chromosome, walk the aligned human sequence position by position:

- If the focal character is a nucleotide (A/T/C/G/N), store the corresponding out-group character at the focal coordinate.
 - If the focal character is a gap (-), skip (insertion in the outgroup, not in focal coordinates).
- (c) Write the resulting sequence as a FASTA file.

```

1 seq = bytearray(b"N" * chrom_length)
2 with open(axt_path, "rt") as f:
3     for block in parse_axt_blocks(f):
4         if block.target_chrom == target_chrom:
5             pos = block.start
6             for h, q in zip(block.target_seq, block.query_seq):
7                 if h.upper() in "ATCGN":
8                     seq[pos - 1] = ord(q)
9                     pos += 1
10 return seq.decode("ascii")

```

Listing 3: Core projection logic (simplified from `project.py`).

6.3 Output

Phase 1 creates `<work_dir>/projected/<species>/<chrom>.fa` for every outgroup species and chromosome.

Parallelization uses Python's `ProcessPoolExecutor` with one worker per task (species $\times$ chromosome).

7 Phase 2: Ancestral State Inference

7.1 The Generalized Algorithm

For each genomic position, the algorithm performs two majority votes and then compares:

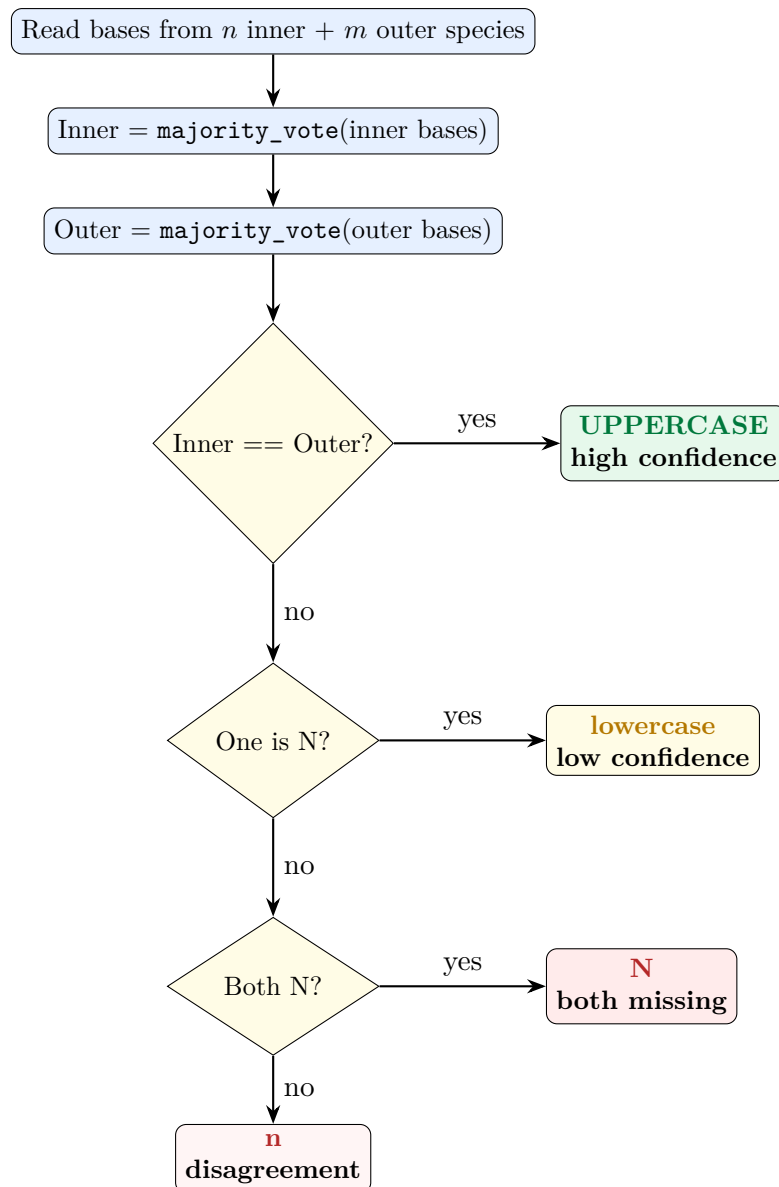


Figure 2: Decision flowchart for the generalized ancestral calling algorithm.

7.2 Core Function

```

1 def call_ancestral_base(inner_bases, outer_bases,
2                           min_inner_freq=1, min_outer_freq=1):
3     inner = majority_vote(inner_bases, min_inner_freq)
4     outer = majority_vote(outer_bases, min_outer_freq)
5
6     if inner != "N" and inner == outer:
7         return inner # HIGH confidence (uppercase)
8     if inner == "N" and outer != "N":
9         return outer.lower() # LOW confidence (lowercase)
10    if inner != "N" and outer == "N":
11        return inner.lower() # LOW confidence (lowercase)
12    if inner == "N" and outer == "N":
13        return "N" # Both missing
14    return "n" # Disagreement

```

Listing 4: `call_ancestral_base()` from `ancestral.py`.

7.3 The majority_vote() Function

```

1 VALID_ALLELES = ("A", "C", "G", "T")
2
3 def majority_vote(bases, min_freq=1):
4     upper = [b.upper() for b in bases]
5     counts = {a: upper.count(a) for a in VALID_ALLELES}
6     best = max(VALID_ALLELES, key=lambda a: counts[a])
7     if counts[best] < min_freq:
8         return "N"
9     return best

```

Listing 5: majority_vote() from utils.py.

Ties are broken in alphabetical order (A, C, G, T) for reproducibility. The function works for any number of input species ($n \geq 1$).

7.4 Output Encoding

Table 3: Confidence encoding in the output FASTA.

Character	Confidence	Condition
A, C, G, T	High	Inner and outer outgroups agree
a, c, g, t	Low	Only one outgroup tier has data
n (lowercase)	Unresolved	Inner and outer disagree
N (uppercase)	Missing	Both tiers lack data

Why Letter Case?

Using uppercase for high-confidence and lowercase for low-confidence calls is a compact encoding within standard FASTA format. Downstream tools can filter by confidence level: for stringent analyses, restrict to uppercase bases (inner–outer agreement); for maximum coverage, include lowercase as well. This convention is compatible with the Ensembl EPO ancestral sequences.

8 Phase 3: Evaluation

8.1 Available Metrics

The evaluation module computes three categories of metrics (the latter two are optional):

Coverage statistics (always): proportion of high-confidence, low-confidence, and missing positions.

Reference comparison (optional): agreement rate between the pipeline’s calls and a reference ancestral sequence (e.g. Ensembl EPO).

VCF comparison (optional): concordance of ancestral calls with REF/ALT alleles at known variant sites.

8.2 Output

For each chromosome, a summary file is written to `<output_dir>/evaluation/<chrom>.evaluation.txt` with sections:

```

1 [coverage]
2   total_positions: 248956422
3   high_confidence: 197145832
4   low_confidence: 25203651
5   missing: 26606939
6   prop_nonmissing: 0.8931
7   prop_high_confidence: 0.7918
8
9 [reference_comparison]
10  test_nonmissing: 0.7912
11  ref_nonmissing: 0.9016
12  both_nonmissing: 5234112
13  agreement_rate: 0.9992
14  disagreement_rate: 0.0008
15
16 [vcf_comparison]
17  num_variants: 6102435
18  prop_nonmissing: 0.8965
19  matches_ref: 0.9453
20  matches_alt: 0.0494
21  matches_either: 0.9948

```

Listing 6: Example evaluation output.

9 Command-Line Interface

9.1 Available Commands

Table 4: CLI commands.

Command	Description
<code>ancify init [-o FILE]</code>	Generate a template configuration file.
<code>ancify project -c CONFIG</code>	Run Phase 1 (coordinate projection).
<code>ancify call -c CONFIG</code>	Run Phase 2 (ancestral calling).
<code>ancify evaluate -c CONFIG</code>	Run Phase 3 (evaluation).
<code>ancify run -c CONFIG</code>	Run all phases end-to-end.

9.2 Global Options

```

1 ancify -v run -c config.yaml      # verbose (debug) logging
2 ancify run -c config.yaml -n 8    # override num_cpus to 8

```

The package can also be invoked as a module:

```

1 python -m ancify run -c config.yaml

```

10 Worked Example: Human (hg38) BCGM

This section walks through the original use case that motivated the package: polarizing the human genome using bonobo, chimpanzee, gorilla, and macaque.

10.1 Phylogenetic Context

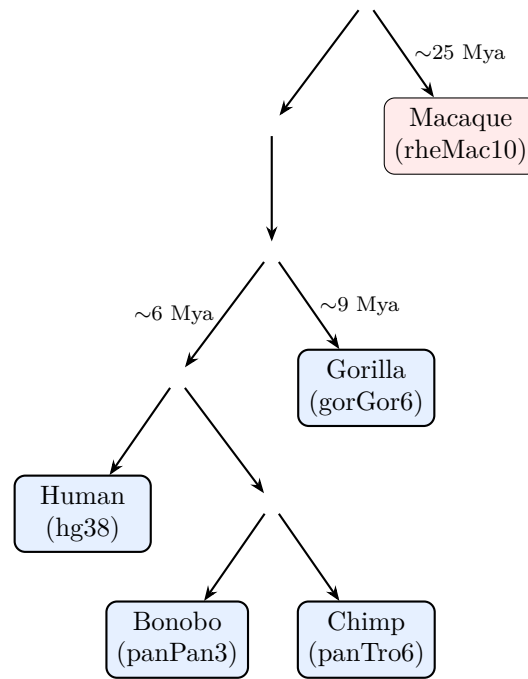


Figure 3: Great ape phylogeny for the human BCGM pipeline.

10.2 Configuration

The example config is provided in `example_configs/hg38_bcgм.yaml`:

```

1 focal_species: human
2 chromosome_lengths: chromoLens.txt
3
4 outgroups:
5   inner:
6     - name: bonobo
7       alignment: hg38.panPan3.net.axt.gz
8     - name: chimp
9       alignment: hg38.panTro6.net.axt.gz
10    - name: gorilla
11      alignment: hg38.gorGor6.net.axt.gz
12   outer:
13     - name: macaque
14       alignment: hg38.rheMac10.net.axt.gz
15
16 output_dir: ./human_ancestor_bcgм
17 num_cpus: 24
18
19 evaluation:
20   reference_dir: ./embi_homo_sapiens_ancestor_GRCh38
21   reference_pattern: "homo_sapiens_ancestor_{chrom_id}.fa"
22   vcf_dir: /path/to/1000genomes/

```

```
23 vcf_pattern: "ALL.chr{chrom_id}...vcf.gz"
```

Listing 7: Key sections of the hg38 BCGM config.

10.3 Input Data

Table 5: Input alignment files for the human BCGM pipeline.

File	Species	Size
hg38.panTro6.net.axt.gz	Chimpanzee	1.67 GB
hg38.panPan3.net.axt.gz	Bonobo	1.66 GB
hg38.gorGor6.net.axt.gz	Gorilla	1.66 GB
hg38.rheMac10.net.axt.gz	Macaque	1.40 GB

These are downloaded from the UCSC Genome Browser:

```
1 wget https://hgdownload.soe.ucsc.edu/goldenPath/hg38/\
2     vsPanTro6/hg38.panTro6.net.axt.gz
```

10.4 Running the Pipeline

```
1 ancify run -c example_configs/hg38_bcgм.yaml
```

Or phase by phase:

```
1 ancify project -c hg38_bcgм.yaml      # 2-8 hours
2 ancify call    -c hg38_bcgм.yaml      # 30-60 minutes
3 ancify evaluate -c hg38_bcgм.yaml      # 20-40 minutes
```

10.5 Evaluation Results

The BCGM calls have been evaluated against the Ensembl EPO (13-primate) ancestral reference:

Table 6: BCGM vs. Ensembl EPO for selected human chromosomes (high-confidence only).

Metric	chr1	chr22	chrX
Non-missing (BCGM)	79.1%	74.6%	44.4%
Non-missing (EMBI)	90.2%	81.2%	44.1%
Disagreement rate	0.083%	0.109%	3.51%
BCGM matches REF/ALT	99.6%	99.5%	99.3%
EMBI matches REF/ALT	99.5%	99.5%	100.0%

Key Observations

- BCGM and EMBI agree at >99.9% of autosomal sites where both call.
- EMBI has higher coverage because it uses a 13-species multiple alignment.
- Including low-confidence calls raises BCGM coverage from ~79% to ~90% on chr1

- (with disagreement increasing from 0.08% to 0.57%).
- Both methods achieve >99.3% concordance with 1000G REF/ALT alleles.

11 Adapting to Other Species

11.1 What You Need

To polarize a new focal species you need:

1. **A reference genome assembly** for the focal species.
2. **A chromosome-lengths file**: a tab-separated file with at least two columns (chromosome name, length in bp).
3. **Pairwise net-AXT alignment files** for each outgroup species. These are widely available from the UCSC Genome Browser’s download site for many vertebrate and insect assemblies.
4. **Phylogenetic knowledge** to decide which species are “inner” (closely related) and which are “outer” (distantly related).

11.2 Choosing Inner vs. Outer Outgroups

Design Guidelines

- **Inner outgroups** should diverge from the focal species *more recently* than the outer outgroup. They provide the primary ancestral signal.
- **Outer outgroups** should be *far enough* that convergent substitutions with the inner outgroup are rare. A divergence of $>2\times$ the inner divergence is a useful rule of thumb.
- Using ≥ 2 inner outgroups enables majority voting, which is robust to single-species alignment gaps or lineage-specific substitutions.
- Multiple outer outgroups can also be used; they undergo their own majority vote.

11.3 Example: Mouse

```

1 focal_species: mouse
2 chromosome_lengths: mm39.chromLens.txt
3
4 outgroups:
5   inner:
6     - name: rat
7       alignment: mm39.rn7.net.axt.gz
8   outer:
9     - name: rabbit
10      alignment: mm39.oryCun2.net.axt.gz
11
12 output_dir: ./mouse_ancestral
13 num_cpus: 8

```

Listing 8: Hypothetical mouse configuration.

11.4 Example: *Drosophila melanogaster*

```

1 focal_species: drosophila_melanogaster
2 chromosome_lengths: dm6.chromLens.txt
3
4 chromosomes: [2L, 2R, 3L, 3R, 4, X]
5
6 outgroups:
7   inner:
8     - name: simulans
9       alignment: dm6.droSim2.net.axt.gz
10    - name: sechellia
11      alignment: dm6.droSec1.net.axt.gz
12   outer:
13     - name: yakuba
14       alignment: dm6.droYak3.net.axt.gz
15
16 output_dir: ./dmel_ancestral
17 num_cpus: 6

```

Listing 9: Hypothetical *Drosophila* configuration.

Note that the chromosome names (2L, 3R, etc.) do not have a `chr` prefix—the package handles any naming convention.

11.5 Creating a Chromosome-Lengths File

For any UCSC assembly, chromosome sizes can be obtained with:

```

1 mysql --user=genome --host=genome-mysql.soe.ucsc.edu -A \
2   -e "SELECT chrom, size FROM chromInfo" dm6 > dm6.chromLens.txt

```

Or from a FASTA index:

```

1 samtools faidx reference.fa
2 cut -f1,2 reference.fa.fai > chromLens.txt

```

12 Using the Output Downstream

12.1 Looking Up an Ancestral Allele

```

1 from ancify.utils import read_fasta
2
3 _, seq = read_fasta("ancestral_calls/chr1.fa")
4
5 position = 1000000 # 1-based
6 allele = seq[position - 1]
7
8 print(f"Ancestral: {allele}")
9 print(f"High confidence: {allele.isupper() and allele in 'ACGT'}")

```

Listing 10: Python: look up ancestral allele at a genomic position.

12.2 Polarizing a VCF File


```

1 for variant in vcf:
2     anc = ancestral_seq[variant.POS - 1]
3
4     if anc.upper() in "ACGT":
5         if anc.upper() == variant.REF:
6             variant.INFO["AA"] = variant.REF    # REF is ancestral
7         elif anc.upper() == variant.ALT:
8             variant.INFO["AA"] = variant.ALT    # ALT is ancestral
9         else:
10            variant.INFO["AA"] = "."             # no match
11     else:
12        variant.INFO["AA"] = "."                # missing
13
14    confidence = "high" if anc.isupper() else "low"

```

Listing 11: Pseudocode for VCF polarization.

12.3 Filtering by Confidence

```

1 # High confidence only
2 if anc.isupper() and anc in "ACGT":
3     use_variant()
4
5 # Including low confidence
6 if anc.upper() in "ACGT":
7     use_variant()
8
9 # Skip unresolved disagreements
10 if anc.lower() == "n":
11     skip_variant()

```

Listing 12: Confidence-level filtering.

12.4 Programmatic API

The package can be used as a library, not just via the CLI:

```

1 from ancify.config import load_config
2 from ancify.project import run_projection
3 from ancify.ancestral import run_ancestral_calling
4 from ancify.ancestral import call_ancestral_base
5
6 # Run the full pipeline
7 cfg = load_config("config.yaml")
8 run_projection(cfg)
9 run_ancestral_calling(cfg)
10
11 # Or call the core function directly
12 base = call_ancestral_base(
13     inner_bases=["A", "A", "G"],    # 3 inner species
14     outer_bases=["A"],              # 1 outer species
15 )
16 # Returns "A" (high confidence: inner majority A == outer A)

```

Listing 13: Using the package programmatically.

13 Theoretical Considerations

13.1 Misinference from Incomplete Lineage Sorting

When ancestral population sizes were large relative to the time between speciation events, gene trees can disagree with the species tree (**ILS**). In such regions, the majority vote among inner outgroups may reflect the derived state. The outer outgroup serves as a safeguard: disagreement between tiers is flagged as **n** (unresolved).

ILS is more common in species with:

- Large ancestral N_e (effective population size).
- Short inter-node intervals on the species tree.
- Rapid radiations.

For the human–chimp–gorilla clade, ILS affects $\sim 1\text{--}3\%$ of the genome.

13.2 Back Mutations and Convergence

A substitution along the outer outgroup lineage creates a false disagreement. Convergent mutations in both the outer and focal lineages create false agreement on the wrong ancestral state. Both events are rare ($\ll 1\%$ of sites for typical mammalian substitution rates) but contribute to the $\sim 0.1\%$ disagreement rate observed in practice.

13.3 Alignment Quality

The pipeline’s accuracy depends fundamentally on alignment quality. Net alignments mitigate paralog confusion by selecting one-to-one best chains, but regions with segmental duplications, transposable elements, or structural variants remain challenging and typically appear as **N** (missing) in the projected sequences.

13.4 The `min_inner_freq` / `min_outer_freq` Parameters

Increasing these thresholds enforces stricter consensus requirements:

Table 7: Effect of `min_inner_freq` with 3 inner outgroups.

Value	Meaning
1	Any single inner species suffices (maximum coverage, lower stringency).
2	At least 2 of 3 must agree (balances coverage and accuracy).
3	All 3 must agree (maximum stringency, reduced coverage).

14 Troubleshooting

MemoryError during Phase 2 Each chromosome loads $n + m$ full-length FASTA sequences. Reduce `num_cpus` to limit concurrent workers, or process chromosomes sequentially.

AXT parsing errors Verify file integrity with `gzip -t file.axt.gz`. Ensure files are standard UCSC net-AXT format.

Sequence length mismatch All projected FASTA files for the same chromosome must have identical length. This can fail if AXT files were generated against different genome builds.

Missing evaluation data Phase 3 is optional. If reference or VCF data are unavailable, omit the evaluation block in the config.

ImportError: scikit-allel VCF evaluation requires `scikit-allel`: install with `pip install 'ancify[evaluate]'`.

15 Complete Module Reference

Table 8: Module and function reference.

Module / Function	Purpose
<code>ancify.config</code>	
<code>load_config(path)</code>	Load and validate a YAML config file.
<code>PipelineConfig</code>	Dataclass holding all pipeline settings.
<code>ancify.utils</code>	
<code>read_fasta(path)</code>	Read a single-record FASTA file.
<code>write_fasta(path, hdr, seq)</code>	Write a single-record FASTA file.
<code>read_chromosome_lengths(p)</code>	Parse a tab-separated lengths file.
<code>majority_vote(bases, min)</code>	Return majority nucleotide or N.
<code>chrom_id(chrom)</code>	Strip leading chr prefix.
<code>ancify.project</code>	
<code>project_alignment(...)</code>	Project one chromosome from AXT to FASTA.
<code>run_projection(config)</code>	Run Phase 1 for all species/chroms.
<code>ancify.ancestral</code>	
<code>call_ancestral_base(...)</code>	Infer ancestral allele at one position.
<code>run_ancestral_calling(cfg)</code>	Run Phase 2 for all chromosomes.
<code>ancify.evaluate</code>	
<code>compute_coverage_stats(seq)</code>	Coverage and confidence breakdown.
<code>compare_to_reference(...)</code>	Agreement with reference ancestral.
<code>compare_to_vcf(...)</code>	Concordance with VCF REF/ALT.
<code>run_evaluation(config)</code>	Run Phase 3 for all chromosomes.
<code>ancify.cli</code>	
<code>main()</code>	Entry point for the <code>ancify</code> command.

A Quick Reference Card

Pipeline Quick Reference

Install:

```

1 pip install . # core
2 pip install '.[evaluate]' # with evaluation deps

```

Generate template config:

```

1 ancify init -o config.yaml

```

Run all phases:

```

1 ancify run -c config.yaml

```

Run individual phases:

```

1 ancify project -c config.yaml
2 ancify call -c config.yaml
3 ancify evaluate -c config.yaml

```

Output encoding:
ACGT = high confidence | acgt = low confidence | n = disagreement | N = missing

B Comparison: General Package vs. Original Scripts

Table 9: What changed from the original hg38-specific scripts to the general package.

Aspect	Original Scripts	<i>ancify</i> Package
Species support	Human only (hg38)	Any focal species
Outgroup count	3 inner + 1 outer (fixed)	n inner + m outer (configurable)
Configuration	Hardcoded paths	YAML config file
Execution	Separate scripts + GNU Parallel	Single CLI with subcommands
Parallelization	Ray / GNU Parallel	<code>ProcessPoolExecutor</code> (stdlib)
Installation	Copy scripts	<code>pip install .</code>
Chromosome naming	<code>chr1</code> – <code>chrY</code> hardcoded	Any naming convention
Evaluation	Human-specific (EMBI + 1000G)	Optional, configurable patterns
Memory (Phase 1)	Dict per position	Bytearray (much more efficient)